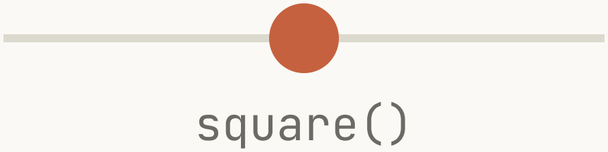
# LESSON 17

# 写一次  用多次



## 函数定义与调用

学完以后，你能把一段计算起个名字，想用的时候喊一声名字就行。

# 同一段计算，为什么要写好几遍?

again.cpp（只阅读）

```
7       cout << a * a << endl;
8       cout << b * b << endl;
9       cout << c * c << endl;
```

算三个数的平方，同样的事写了三遍。

要是计算更长，比如判断闰年，写三遍就容易抄错，改一处还得改三处。

办法是：把这段计算**写一次，起个名字**，要用时喊它的名字。

这样起了名字的一段程序，叫**函数**。

# 定义一次，在 main 里调用

square.cpp

```
 4 int square(int x) {
 5     return x * x;
 6 }
 8 int main() {
 9     int n = 0;
10     cin >> n;
11     int result = square(n);
12     cout << result << endl;
```

第 4 到 6 行是**定义**：写在 main 前面，告诉程序 square 怎么算。

第 11 行是**调用**：把 n 交给 square，算出来的结果放进 result。

输入 3，屏幕上

9

# 一个函数的四个部分

```
int square(int x) { return x * x; }
```

| | |
|---|---|
| `int` | 交回来的结果是什么类型 |
| `square` | 函数的名字，自己起，用英文 |
| `(int x)` | 参数：调用时交进来的数，在函数里叫 `x` |
| `return` | 把结果交回给调用它的地方 |

`main` 也是一个函数：程序从 `main` 开始运行。

# 输入 3：调用时发生了什么

| | | |
|---|---|---|
| 第 11 行 | `square(n)`，n 是 3 | 跳到 square 里 |
| 第 4 行 | x 拿到 3 | 参数接住交进来的数 |
| 第 5 行 | `return 3 * 3` | 把 9 交回去 |
| 回到第 11 行 | `square(n)` 变成 9 | result 是 9，第 12 行输出 |

调用就像把题交给一个专门算平方的同学：你给他 3，他交回 9。

交回来以后，程序接着从调用的地方往下走。

# 改一改：交进去的数

square.cpp

```cpp
#include <iostream>
using namespace std;
int square(int x) {
    return x * x;
}
int main() {
    int n = 3;
    cout << square(n) << endl;
    return 0;
}
```

square.cpp

第 8 行输出 square(n)

**9**

点黄色的数字，改完结果马上变。先猜再改：5、10、0。

函数只写了一次，交进去什么数，就交回什么数的平方。

# 小测：main 里没调用 square，会输出 9 吗？

square 定义好了，main 里没有 square(…)，输入 3

1 9，然后 Hi

2 只有 Hi

3 报错

4 什么也没有

答对了！定义只是说“怎么算”，main 里调用了才会算。

# 小测：cout << square(square(2)); 输出几？

1 4

2 8

3 16

4 报错

答对了！先算里面：square(2) 交回 4，再算 square(4)，交回 16。

# 动手改一改

从 square 开始。函数名和算式都要改，调用的地方也别忘了。

## 写一个 cube

读入整数 n，是 -100 到 100。写函数 cube 返回 n 的三次方，调用后输出。

| 输入 2 | 输入 -3 |
| --- | --- |
| 8 | -27 |

Dev-C++：打开 starter/02_cube.cpp

# 两个参数，用逗号隔开

diff.cpp

```cpp
int diff(int a, int b) {
    return a - b;
}
int main() {
    int a = 0, b = 0;
    cin >> a >> b;
    cout << diff(a, b) << endl;
```

调用时交进去的数，按**顺序**一个对一个：第一个给 a，第二个给 b。

结果不一定先放进变量，可以直接拿去输出。

输入 7 2

**5**

# 猜一猜：写成 diff(b, a)

diff.cpp（只阅读）

```
11      cout << diff(b, a) << endl;
```

函数里算的是 2 - 7。参数认的是**位置**，不看外面的变量叫什么名字。

屏幕上

**-5**

# 动手改一改

从 diff 开始。两个参数换成长和宽。

## 长方形周长

一行读入长 w 和宽 h，都是 1 到 1000 的整数，用空格隔开。写函数 perimeter 返回周长 2 × (w + h)，调用后输出。

输入 3 5

```
16
```

输入 1 1

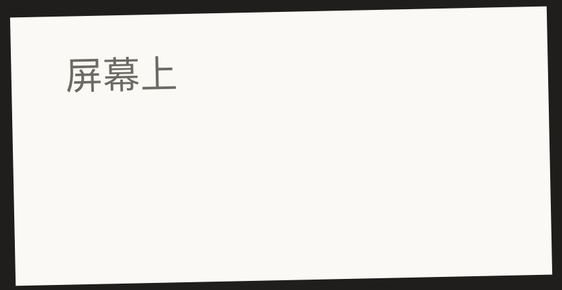
```
4
```

Dev-C++：打开 starter/04_perimeter.cpp

# 猜一猜：只写 square(n);

alone.cpp（只阅读）

```
10      cin >> n;
11      square(n);
```




什么也没有。**return 是交回结果，cout 才是往屏幕上写。**

交回来的 9 要放进变量，或者交给 cout，才看得见。

# 函数里也能放 if：返回大的那个

bigger.cpp

```
4 int bigger(int a, int b) {
5     if (a > b) {
6         return a;
7     }
8     return b;
9 }
```

输入 3 8

8

运行到 return，函数就结束了，后面的不再运行。

a 大就在第 6 行交回 a；否则走到第 8 行交回 b。

# 动手改一改

从 bigger 开始。负数就交回它的相反数，用 -x。

## 绝对值

读入整数 x，是 -1000 到 1000。写函数 absolute 返回 x 的绝对值，调用后输出。

| 输入 -5 | 输入 5 | 输入 0 |
| --- | --- | --- |
| 5 | 5 | 0 |

Dev-C++：打开 starter/06_absolute.cpp

# 读报错：函数写在了 main 后面

square.cpp（只阅读）

```
4 int main() {
5     …
6     int result = square(n);
7     …
8 }
9 int square(int x) { … }
```

程序从上往下读。读到第 7 行时，还没见过 square。

把 square 的定义搬到 main 前面就好。

```
7:18: error: 'square' was not declared in this scope
```

Dev-C++ 显示的文字和位置可能不一样，还是先找行号和 error。

# 找错：哪一行有问题？

bigger.cpp（只阅读）

```
1 int bigger(int a, int b) {
2     if (a > b) {
3         return b;
4     }
5     return b;
6 }
```

这段想返回大的数。

输入 3 8 输出 8，是对的。

输入 8 3 却输出 3。

点你认为有问题的那一行。

点代码里的一行

测试函数时，两个参数换个大小顺序再试一次。

# 自己挑战

# 数一数闰年

起始程序的 main 已经写好：读入 a 和 b（1 到 3000，a 不大于 b），数 a 到 b 年里有几个闰年。你只写函数 bool leap(int y)。

闰年规则

**能被 4 整除但不能被 100 整除，**

**或者能被 400 整除**

输入 2000 2024，输出

**7**

输入 1900 1900，输出

**0**

先自己写，卡住了再翻下一页。

# 卡住了？一次看一条提示

**提示 1**　bool 函数交回 true 或 false，给 main 里的 if 用。

**提示 2**　“能被 4 整除”写成 y % 4 == 0，“不能被 100 整除”写成 y % 100 != 0。

**提示 3**　规则里有“并且”和“或者”，想想第 5 课的 && 和 ||，再加上括号。

# # 造一个自己的工具箱

写一两个自己的函数，在 `main` 里调用它们。用途你自己定。

## 先做到

写一个有参数、有 `return` 的函数，在 `main` 里调用并输出结果。

## 选你喜欢的

温度换算、零花钱翻倍、游戏得分加成、年龄算属相，都可以。

## 还想更厉害

写两个函数，把一个的结果交给另一个。

没有标准答案。写之前说清想要的结果，写完试试最小和最大的输入。

# # 和同桌换着玩

## 1

### 换着运行

运行同桌的程序，先猜函数会交回什么，再看结果

## 2

### 说一个喜欢的

告诉同桌，你最喜欢哪个函数

## 3

### 提一个想法

问问同桌：还可以加一个什么函数？

听完再改一版。想一想、做出来、试一试、分享、再改，程序就是这样越做越好的。

# 说得清吗?

**Q1** **定义和调用有什么不同?**

定义说怎么算，调用才真正去算。只定义不调用，函数不运行。

**Q2** **`return` 和 `cout` 有什么不同?**

`return` 把结果交回调用的地方，`cout` 才往屏幕上写。

**Q3** **`diff(b, a)` 和 `diff(a, b)` 为什么不一样?**

参数按位置一个对一个，不看外面变量的名字。

# 速查卡

```
int square(int x) { return x * x; }
```

| | |
|---|---|
| `square(n)` | 调用：把 n 交进去，换回结果 |
| `return` | 交回结果，函数到这里结束 |
| `(int a, int b)` | 几个参数用逗号隔开，按位置对应 |
| `bool` | 交回 true 或 false 的函数 |
| `写在 main 前面` | 先定义，后调用 |

# # 这些后面再学

### 参数是副本

函数里改了 x，外面的 n 会变吗?

### 作用域

变量在哪里能用，在哪里不能用

### 递归

函数自己调用自己

今天学会函数，一段计算写一次、起个名字，以后喊名字就能用。

# 谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 cpp-for-youth

下一课：函数参数与作用域